

CS 610 Semester 2024–2025-I: Assignment 2

Name: **Vinayak Devvrat**

Roll Number: **241110079**

September 7, 2024

Problem 1

The following table provides the information on the performance bugs by manually analysing the code. The fixes made have also been described.

PERFORMANCE ANALYSIS

Table 1

S No.	Performance Bug	Description	Fix
1	True Sharing due to <code>total_lines_processed</code>	True sharing misses are incurred due to repeated accesses by each thread to this variable.	Since this has to be updated by each thread, these misses are going to occur regardless. Hence, no fix has been provided to mitigate these misses.
2	True Sharing due to <code>total_words_processed</code>	True sharing misses are incurred as each thread updates this variable.	<code>total_words_processed</code> is made atomic using <code>std::atomic</code> . This eliminates the use of a mutex lock. Hence eliminating the true sharing entirely.
3	True Sharing due to <code>word_count_mutex</code>	True sharing misses are incurred as each thread acquires this mutex lock to make atomic update to <code>total_words_processed</code> .	<code>total_words_processed</code> is made atomic using <code>std::atomic</code> . This eliminates the use of a mutex lock. Hence eliminating the true sharing entirely.
4	False Sharing due to <code>thread_count</code> array	False sharing misses are incurred because the entire array gets invalidated and	<code>thread_count</code> array has been modified to a 2D array where the first element of each row is the thread

		thereby invalidating the indices of threads as well.	count and the rest is the necessary padding to make the row reside in one cache line. So if a thread updates, only that thread's row is invalidated. Hence, false sharing misses are eliminated entirely.
--	--	--	---

Additional Fix: `total_lines_processed` and `total_words_processed` are two 8 B variables which would be stored in contiguous locations in a 64 B cache line. We may incur misses in case when two threads are executing different lines of the `thread_runner` function. Hence a padding of `chars` has also been added to have these two variables in different cache lines so that update to one variable does not affect the other.

The following table gives the quantitative comparison using `perf` of the original and improved codes:

COMPARITIVE ANALYSIS USING `perf`
Table 2

S No.	perf Counter	Value in Original Code	Value in Improved Code
1	Load Local HITM	15	4
2	Total Loads	113	16
3	Total Stores	46	2

Complete `perf` reports for each code have been attached.

Speed up

Time take in original code, $T = 9111 \mu s$.

Time take in improved version, $T' = 3828 \mu s$.

Speed up, S , is given by,

$$S = \frac{T}{T'} = \frac{9111}{3828} = \mathbf{2.38}$$

Note: The test cases given were not big enough to generate substantial values for `perf` analysis. Hence, the above reports are generated by running the testcases present in the `test2` directory.

Problem 2

- Conditional Variables

- `buffer_is_full`: To indicate the buffer is full and synchronize the producers and consumers accordingly.
 - `buffer_is_empty`: To indicate the buffer is empty and synchronize the producers and consumers accordingly.
- **Mutex Variables**
 - `lock`: Mutex variable to synchronize access to the shared buffer between the consumer and a producer thread.
 - `exclusive_lock`: Mutex variable shared between the various producer threads. This lock is not released until the thread has read L lines or the file has ended.
- **Other Global Variables**
 - `START_READING_FROM`: Holds the last line read by the last thread to access the input file.
 - `END_OF_FILE_REACHED`: Initialised to 0. It is set to 1 the end of file has been reached.
 - `buffer`: A standard queue that implements the buffer.
 - **Producer Function**
 - `read_from_file`: This implements the reading of the file. A thread acquires the mutex `lock`. It waits if the `buffer` is full. If `END_OF_FILE_REACHED` is set, then the thread breaks from the loop and exits. Otherwise, it writes its share of L lines completely or until the `buffer` is full in which case it waits for `buffer` to be empty before resuming in case it has not read L lines.
 - **Consumer Function**
 - `write_to_file`: This implements the writing to the output file. The consumer thread acquires the mutex `lock`. It waits if the `buffer` is empty. If `PRODUCERS_COMPLETED` is set, then the thread breaks from the loop and exits. Otherwise, it reads all the lines present in the buffer.

The code has been attached. Comments are added alongside the lines as explanation.

Use the following to compile: `g++ prob2.cpp -o prob2 -pthread`

Use the following to run (as an example): `./prob2 bigfile.txt 15 10 14 output.txt`

Problem 3

1. $A(i, j-i)$ and $A(i, j-i-1)$

$$i_0 = i_0 + \Delta i \implies \Delta i = 0$$

$$j_0 - i_0 = (j_0 + \Delta j) - (i_0 + \Delta i) - 1 \implies j_0 - i_0 = j_0 + \Delta j - i_0 - \Delta i \implies \Delta j = 1$$

Distance vector = (0,1); Direction vector = (=,+)

This indicates **flow dependence**.

There are no anti and output dependences in this case.

2. $A(i, j-i)$ and $A(i+1, j-i)$

$$i_0 = (i_0 + \Delta i) + 1 \implies \Delta i = -1$$

$$j_0 - i_0 = (j_0 + \Delta j) - (i_0 + \Delta i) \implies j_0 - i_0 = j_0 + \Delta j - i_0 - (-1) \implies \Delta j = -1$$

Distance vector = (-1,-1); Direction vector = (-,-)

This indicates **no flow dependence**.

There is a **anti-dependence** here.

There is no **output dependence** in this case.

3. $A(i, j-i)$ and $A(i-1, i+j-1)$

$$i_0 = (i_0 + \Delta i) - 1 \implies \Delta i = 1$$

$$j_0 - i_0 = (i_0 + \Delta i) + (j_0 + \Delta j) - 1 \implies j_0 - i_0 = i_0 + 1 + j_0 + \Delta j - 1 \implies \Delta j = -2i$$

Distance vector = (1,0); Direction vector = (+,=)

This indicates **flow dependence**.

There are no **anti** and **output** dependences.